

Python for-else Loops

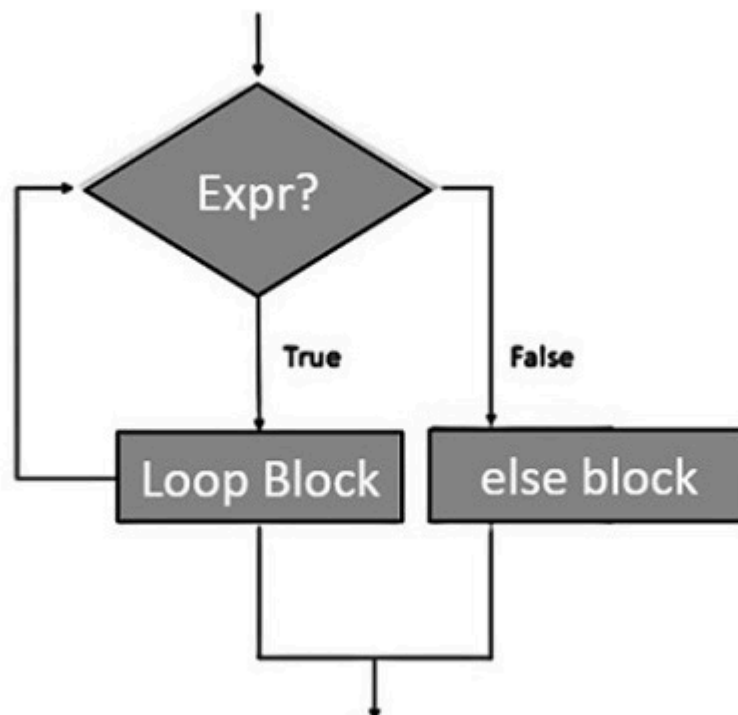
Python - For Else Loop

Python supports an optional **else block** to be associated with a **for loop**. If a **else block** is used with a **for loop**, it is executed only when the for loop terminates normally.

The for loop terminates normally when it completes all its iterations without encountering a **break statement**, which allows us to exit the loop when a certain condition is met.

Flowchart of For Else Loop

The following flowchart illustrates use of **for-else loop** –



Syntax of For Else Loop

Following is the syntax of for loop with optional else block –

```

for variable_name in iterable:
    #stmts in the loop
    .
    .
    .
else:
    #stmts in else clause
    .
    .

```

Example of For Else Loop

The following example illustrates the combination of an else statement with a for statement in **Python**. Till the count is less than 5, the iteration count is printed. As it becomes 5, the print statement in else block is executed, before the control is passed to the next statement in the main program.



```

for count in range(6):
    print ("Iteration no. {}".format(count))
else:
    print ("for loop over. Now in else block")
print ("End of for loop")

```

On executing, this code will produce the following **output** –

```

Iteration no. 1
Iteration no. 2
Iteration no. 3
Iteration no. 4
Iteration no. 5
for loop over. Now in else block
End of for loop

```

For-Else Construct without break statement

As mentioned earlier in this tutorial, the else block executes only when the loop terminates normally i.e. without using break statement.

Example

In the following program, we use the for-else loop without break statement.

```
for i in ['T','P']:
    print(i)
else:
    # Loop else statement
    # there is no break statement in for loop, hence else part gets executed
    directly
    print("ForLoop-else statement successfully executed")
```

On executing, the above program will generate the following output

```
T
P
ForLoop-else statement successfully executed
```

For-Else Construct with break statement

In case of forceful termination (by using break statement) of the loop, else statement is overlooked by the interpreter and hence its execution is skipped.

Example

The following program shows how else conditions work in case of a break statement.

```
for i in ['T','P']:
    print(i)
    break
else:
    # Loop else statement
    # terminated after 1st iteration due to break statement in for loop
    print("Loop-else statement successfully executed")
```

On executing, the above program will generate the following output

T

For-Else with break statement and if conditions

If we use **for-else** construct with **break statement** and **if condition**, the **for loop** will iterate over the iterators and within this loop, you can use an **if block** to check for a specific condition. If the loop completes without encountering a **break statement**, the code in the else block is executed.

Example

The following program shows how else conditions works in case of break statement and conditional statements.

```
# creating a function to check whether the list item is a positive
# or a negative number
def positive_or_negative():
    # traversing in a list
    for i in [5,6,7]:
        # checking whether the list element is greater than 0
        if i>=0:
            # printing positive number if it is greater than or equal to 0
            print ("Positive number")
        else:
            # Else printing Negative number and breaking the loop
            print ("Negative number")
            break
    # Else statement of the for loop
    else:
        # Statement inside the else block
        print ("Loop-else Executed")
# Calling the above-created function
positive_or_negative()
```

On executing, the above program will generate the following output

Positive number
Positive number
Positive number
Loop-else Executed

TOP TUTORIALS

[Python Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[C Programming Tutorial](#)
[C# Tutorial](#)
[PHP Tutorial](#)
[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)
[Amazon Web Services Tutorial](#)
[Microsoft Azure Tutorial](#)
[Git Tutorial](#)
[Ethical Hacking Tutorial](#)
[Docker Tutorial](#)
[Kubernetes Tutorial](#)
[DSA Tutorial](#)
[Spring Boot Tutorial](#)
[SDLC Tutorial](#)
[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)



Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.